Manual del Programador — Piedra, Papel o Tijera v1.0.0

Actividad 7 · LPR 5.º 3.ª A-B · Autor: Alexis Maier

1. Propósito

Describir la arquitectura del juego y de su capa de persistencia en disco.

2. Requisitos y compilación

- `g++` con soporte C++11 (MinGW-w64 / MSYS2).
- `g++ -std=c++11 src/main.cpp -o src/piedrapapelotijera.exe`
- Ejecutar desde la raíz del repositorio (rutas relativas a `src/`).

3. Modelo de datos

```
struct Partida {
     int idPartida;
     string jugador;
     string eleccionJugador;
     string eleccionIA;
     string resultado;
};
```

4. Funciones

- `main()`: inicializa `srand(time(0))`, muestra el menú en un `do...while` hasta que se elige 3.
- `jugarPartida(int& contadorID)`: crea la `Partida`, lee el nombre (con `cin.ignore()` + `getline`), pide la jugada, genera la elección de la IA con `rand() % 3 + 1`, evalúa el resultado y llama a `guardarEnDisco`.
- `guardarEnDisco(const Partida& p)`: abre `src/historial_partidas.txt` con `ofstream(..., ios::app)` y escribe una línea CSV `id,jugador,eleccionJugador,eleccionIA,resultado`.
- `leerHistorialDisco()`: abre el archivo con `ifstream`, lee cada campo con `getline(archivo, campo, ',')` y muestra las partidas y el total.

5. Persistencia

La RAM es volátil: al cerrar el programa se pierden las variables. El archivo `.txt` en el disco es no volátil, por lo que el historial sobrevive entre ejecuciones. El modo `ios::app` agrega al final sin borrar lo anterior; sin él, cada partida sobrescribiría el archivo.

6. Limitaciones conocidas

- `contadorID` vuelve a 1 en cada ejecución, por lo que los ID se repiten entre sesiones.
- Un nombre con comas rompería el formato CSV.
- La jugada inválida (distinta de 1 o 2) se interpreta como Tijera.